©2026 Tucson Amateur Packet Radio Corporation




# Introduction

The MVR ("Minimum Viable Reference") GPSDO is a low-cost GPS disciplined oscillator intended primarily for HF radio applications. It has a 10 MHz output as well as three additional outputs that are programmable to frequencies between about 200 kHz and 200 MHz (operation outside these ranges is possible, but not guaranteed). Thus, it can serve as an external clock source for SDRs such as the RX888, WEB-888, or KiwiSDR.

The MVR is powered from a USB-C connector which is also used to access the configuration menu. Once the unit is configured, it can operate from a standard USB-C power supply; the computer does not need to be connected except when changing configuration. The antenna connector provides 3.3 volts to drive a GPS antenna. The outputs are square waves at about +10 dBm into 50 ohms.

The MVR's long term accuracy approaches that of the GNSS satellite constellation, but its low-cost design goal means that its short-term stability is not as good as some other GPSDOs. To be more specific, its 10 MHz output wiggles from second to second by about 9 milliHertz. For applications below 30 MHz, this is usually lost in the noise. But frequency and phase noise scale with frequency, so at VHF or UHF frequencies, the short-term noise may become an issue. Other instruments are a better choice for such applications.

Another limitation of the MVR is that it does not have a "holdover" capability – if GNSS signal is lost for more than a second or so, the unit will lose lock and the frequency will shift significantly. However, with multiple satellite constellations available today, loss of lock should be very rare unless the antenna is severely compromised, or there is active GNSS jamming in the area – in which case you probably have bigger problems to deal with!

See the "Performance" section for further information on the MVR's frequency stability and performance.

# Operation

Once configured, the MVR requires no user interaction in normal use. Connect a GNSS antenna to the ANT connector on the rear, and a source of 5V power to the USB-C connector. The receiver provides 3.3 volts to power the antenna's low noise amplifier.

The two LEDs visible through the rear panel may flash briefly, and should then turn off. After about 45 seconds, the GNSS LED should light, indicating that the GNSS receiver is locked. Then after about two minutes the PLL LED should light, indicating that the PLL has locked and that the output frequencies are now stabilized.

After a cold start, it takes about 30 minutes for the unit's internal temperature to stabilize for best short-term stability. In general, it's best to keep the MVR in a stable temperature environment during operation. The actual temperature isn't critical, but keeping the unit away from drafts and sudden temperature changes will improve stability.

## Initial Firmware Load

You need to load the MVR firmware onto the XIAO processor module inside the MVR before you use the device. Go to **https://github.com/TAPR/mvr_firmware/releases/latest** and download the file with the ".uf2" extension to your computer.

The filename has the format **mvr_firmware_YYYYMMDD.P.ino.uf2** – where **YYMMDD** is the date of the release, and **P** is the point number. For example, **mvr_firmware_20261008.1.ino.uf2**.

Firmware loading is done by placing the XIAO into its bootloader mode. On a new XIAO that has not had the MVR firmware uploaded before, you need to do this manually.

After you've assembled the MVR, but before you put it in the enclosure, load the firmware by following these steps.

1. Don't connect the USB cable to the computer yet.

2. Find the tiny "BOOT" button on the XIAO module. It is on the end of the module toward the front of the MVR, opposite the USB connector. The button is to the right of the LED on that end of the module.

3. While holding the BOOT button, plug the USB cable into your computer. It should recognize the XIAO as a USB drive named "RPI-RP2".

4. When your computer recognizes the XIAO as a drive, release the BOOT button.

5. Drag and drop the .uf2 file onto the RPI-RP2 drive. The XIAO will reboot with the new firmware running.

After the initial firmware upload, future uploads can be done using a menu option as described below, without the need for pressing the BOOT button.

## Communicating with the MVR

You can configure the MVR and monitor its status using a serial terminal like PuTTY in Windows or minicom or tio in Linux. Since the connection is native USB, the configured baud rate shouldn't matter; if you are required to specify a speed, select 115200. There is no flow control, so a simple terminal configuration will work.

### Startup Sequence

At power on, the firmware prints the startup progress:

```
=== MVR GPSDO B Firmware v20261008.1 (built Oct  8 2026 10:15:52) ===

GPS02-UBX Configuration
  TP when locked : 25000 Hz, 50% duty
  TP when unlocked: 0% duty (output suppressed)
Attempt 1 of 5
Step 1: Configure timepulse (UBX-CFG-TP5)
  [OK  ] TP1: unlocked (0% duty) -> locked timepulse  (ACK)
Step 2: C/N0 input filter for fast unlock detection (UBX-CFG-VALSET)
  [OK  ] C/N0 threshold for fast unlock detection  (ACK)
Step 3: Enable NMEA navigation messages on UART1 (UBX-CFG-MSG)
  [OK  ] GGA  (Fix data)  (ACK)
  [OK  ] GLL  (Lat/Lon)  (ACK)
  [OK  ] GSA  (DOP/active sats)  (ACK)
  [OK  ] GSV  (Sats in view)  (ACK)
  [OK  ] RMC  (Recommended minimum)  (ACK)
  [OK  ] VTG  (Course/speed)  (ACK)
Step 4: Skipped (GNSS_SAVE_TO_FLASH is 0)
GNSS configuration succeeded.

  Loaded saved configuration from flash.

Si5351A Configuration
  I2C address:    0x60
  Reference:      10.000000 MHz on XA (0 pF load cap)
  Drive strength: 8 mA
  PLLA: VCO = 810.000 MHz (integer feedback, FBA_INT set)
  CLK0: requested 27000000 Hz, actual 27000000.000000 Hz, fractional error (0.00
00000000000, 0.000 ppb), PLLA, integer mode
  CLK1: powered down
  CLK2: powered down
Done.

Setup complete. Type any character for the Si5351 menu.
```

Thereafter, it prints a status line every 30 seconds with additional messages for significant events:

```
[96] GPS: GPS fix, 12 sats | Loop: unlocked | CLK0 27000000Hz CLK1 off CLK2 off
[126] GPS: GPS fix, 12 sats | Loop: unlocked | CLK0 27000000Hz CLK1 off CLK2 off
[129] Loop: LOCKED
[156] GPS: GPS fix, 12 sats | Loop: LOCKED | CLK0 27000000Hz CLK1 off CLK2 off
[186] GPS: GPS fix, 12 sats | Loop: LOCKED | CLK0 27000000Hz CLK1 off CLK2 off
```

To get to the menu, press any key. You'll see:

```
=== MVR GPSDO B Firmware v20261008.1 (built Oct  8 2026 10:15:52) ===
  1) Set CLK0 frequency  (currently 27000000 Hz)
  2) Set CLK1 frequency  (currently off)
  3) Set CLK2 frequency  (currently off)
  4) Set drive strength, all channels (currently 8 mA)
  5) Show this menu again
  6) Show lock history
  7) Restore compile-time defaults (overwrites saved config)
  8) Toggle raw NMEA passthrough (currently OFF; boot default OFF)
  9) Reboot into firmware-update (UF2) mode
 99) Exit menu
>
```

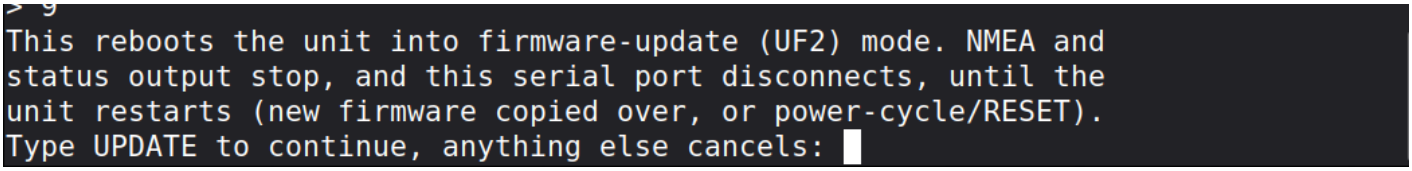
Enter the appropriate number to make a change. For example, to change a clock frequency:

```
> 1
(tip: frequencies that are exact multiples of 10 Hz are
 guaranteed exact synthesis -- see project notes)
Enter new frequency for CLK0 in Hz (2500-200000000), or 0 to power it down: 1000
000

  Requested: CLK0 = 1000000 Hz
  Achieved:  1000000.000000 Hz
  Fractional error: 0.0000000000000  (0.000 ppb)
  Accept this frequency? (y/n): y
  PLLA: VCO = 900.000 MHz (integer feedback, FBA_INT set)
  CLK0: requested 1000000 Hz, actual 1000000.000000 Hz, fractional error (0.0000
000000000, 0.000 ppb), PLLA, integer mode
  CLK1: powered down
  CLK2: powered down
CLK0 set to 1000000 Hz.
```

Note that when you enter the frequency, you will be shown the frequency error (if any) that will result. There are some frequencies that the Si5351A chip cannot generate exactly, so this gives you a warning if the frequency won't be exact. (Note – the MVR firmware uses a "continuing fraction" algorithm that does a

much better job of finding an exact frequency than the default Si5351A libraries you might see in Arduino examples.)




To update the firmware, enter "9" at the menu. You'll be given this prompt:

After typing UPDATE (it's case-insensitive), you should see an RPI-RP2 USB drive appear in your file manager. Drag the new firmware file with.uf2 extension to it, and the board will reboot with the new firmware.

Once you enter UPDATE, the board will not respond to other commands. The only way out is to either upload a firmware file, or pull the USB-C cable and reconnect it. The existing firmware will remain until a new file replaces it, so there is no danger in disconnecting the module to abort firmware upload.

# Circuit Description

The MVR uses a phase-locked loop ("PLL") to steer a 10 MHz crystal oscillator to follow the very precise time signal generated by a GNSS[1] module. The oscillator output is available directly via an SMA connector, and is also fed to an Si5351A clock synthesizer chip that can generate up to three different programmable frequencies. The PCB is four layers, with all critical signals (10 MHz, control voltage, etc.) on inside layers. This is meant to protect them from noise, and also to prevent the 10 MHz signal from escaping to possibly interfere with a receiver tuned to 10 MHz for WWV.

The NiceRF GPS02-UBX GNSS module (U1) outputs a TIMEPULSE[2] signal at 25 kHz, generated by the module's own on-board free-running crystal clock which is pulled into alignment with GNSS time once per second. The TIMEPULSE signal is very precise over the long term, but because of the way it is generated, as well as the inherent short-term noise of the GNSS signal, it has too much jitter to be useful directly for RF applications. (Note: The GPS2-UBX uses a standard u-blox M10 chipset. It's a low-cost component with good performance and is well tuned to this application.)

The 10 MHz VCXO (U10) is a crystal oscillator whose frequency can be controlled by applying a voltage to its electronic frequency control (EFC) pin. It is relatively stable in the short term, but is only approximately at the correct frequency and is subject to temperature, humidity, and other environmental changes, and also will drift in frequency over time. It's clean enough to run a radio, but not accurate enough to be useful for some science applications.

## The Phase Locked-Loop

The purpose of a GPSDO is to combine the VCXO's short term stability with the GNSS's long-term accuracy. It does this through the PLL, which compares the VCXO output to the TIMEPULSE and generates a correction voltage applied to the VCXO's EFC (electronic frequency control) input to steer it to match the phase of the TIMEPULSE.

The PLL uses a 74HC4046 (U6) as a phase detector. It has two inputs: the 25 kHz TIMEPULSE signal, and a 25 kHz signal divided from the VCXO by U7, a 74HC390 decade divider, and U8, a 74HC393 binary divider. The 4046 outputs a series of pulses as the two signals drift past each other; when they are precisely in phase, the output is essentially zero. These pulses are fed to a low pass filter to turn them into a control voltage to steer the VCXO frequency.

The low pass filter is the most complicated part of the circuit. It has several stages. First, R19, R20, and C20 form the traditional "loop filter" that sets the bandwidth and damping of the PLL and its ability to capture off-frequency signals. However, these components by themselves aren't a strong enough filter to block all the artifacts of the process, and since R19 is 4.7 megohms, this is a very high impedance part of the circuit that can't drive other components.

U9, an OPA391 op amp, serves two purposes. First, it acts as a high-impedance input buffer to terminate the loop filter, and second, it is configured as an active filter to further attenuate spurious signals on the control voltage. After U9 come another two stages of passive filters – R23/C27 and R24/C28. These further clean up the control voltage. Overall, the filter has a bandwidth of about 0.15 Hz and attenuates by more than 90 dB at 30 Hz.

1 "GNSS" stands for Global Navigation Satellite System. We used to talk about "GPS", the U.S. system, but modern receivers can use not only GPS, but also the Russian "GLONASS", the EU "Galileo", and the Chinese "Beidou" systems, so "GNSS" is the more accurate term.

2 This used to be called a "PPS" signal but modern GNSS devices can output a wide range of frequencies apart from one pulse per second.

The VCXO 10 MHz output is buffered into three outputs by U5. One goes to the front panel SMA connector, one goes to the divider mentioned above, and one goes to the Si5351 clock generator.

## Synthesizer

The Si5351A (U4) has internal oscillator components and is designed to have a crystal connected between its XA and XB pins, but it can be driven by applying a signal to the XA pin. R6, R8, and C8 condition the 10 MHz input. The three outputs each go through a small series resistor, a 10 nF capacitor, and an ESD protection diode before reaching the SMA connectors on the front panel.

## Processor Module

The GNSS module and the clock generator both require configuration at power up, and if output frequencies are to be changed. A Seeed Studios XIAO RP2040 module handles this. The module mounts to the MVR circuit board with two 7-pin through-hole headers. It has a USB-C connector that is used both for communication and to power the MVR. The GNSS uses serial (UART) data, while the Si5351A communicates via an I2C bus. The XIAO header provides both these signals. The firmware and its configuration are described above.

## Power Conditioning

We paid a lot of attention to providing clean power for the sensitive components on the board; power supply noise could directly affect the quality of the output.

Power is applied via the XIAO module USB connector. Pin 14 of the module passes the USB 5 volts directly through, while pin 12 provides 3.3 volts from the module's onboard regulator. Both these outputs are used. 3.3 volts from pin 12 drives GNSS module U1 through bypass capacitors and a ferrite bead.

Five volts from pin 14 feeds U11, a low noise, low dropout 3.3 volt regulator. This clean supply drives the rest of the circuitry on the board. Critical components have filtering at their power inputs, including ferrite beads.

## Why 25 kHz?

The choice of a 25 kHz comparison frequency might seem odd. Most traditional GPSDOs divide the oscillator to a pulse-per-second signal and compare that to the PPS from the GNSS. Back when early GPS modules had a fixed PPS output that couldn't be set to a faster rate, there was no other choice. But PPS comparison has two drawbacks for an analog phase-detector loop like this one.

First, it allows for only one correction per second, which provides very coarse time resolution. The loop makes infrequent corrections, so they need to be large and the ability to provide fine frequency control is reduced. Second, and more fundamentally, the timepulse output isn't a perfect 1 Hz edge. It's generated by the GNSS module's free running clock, jerked to align to the top of second. This leaves a periodic quantization artifact (often described as a sawtooth). Comparing at 1 Hz puts the reference frequency and dominant noise source on top of each other, so there's no way to filter one out without also filtering the desired signal.

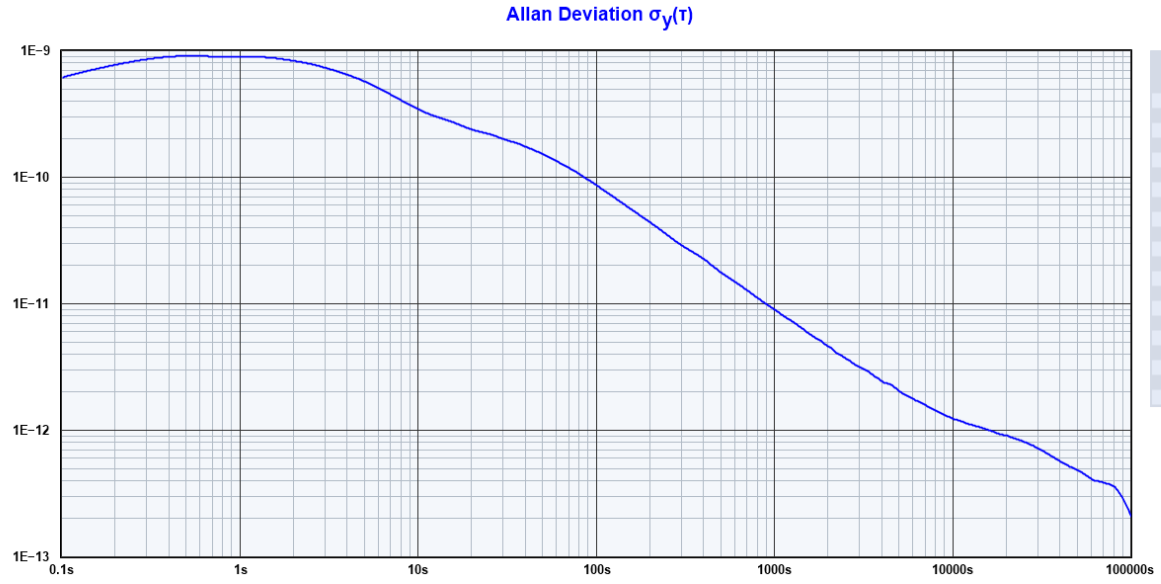
Comparing at the full 10 MHz doesn't work well, either, for a different reason: the gain of the loop scales with how many VCXO cycles occur per comparison. While a 1 PPS loop has extremely low gain, a 10 MHz loop has too much. Taming that much gain down to the required sub-Hz bandwidth requires pushing the loop filter's resistor value into the multi-megohm range. That means the circuit has to operate at extremely high impedance, which causes susceptibility to power noise, current leakage across the PCB, and other difficult-to-tame problems. In bench testing, comparing directly at 10 MHz produced this failure mode: the loop would not stay locked.

25 kHz sits in the sweet spot between these two failure modes. The comparison rate is high enough to give the phase detector fine time resolution and to push reference-related artifacts far above the sub-Hz loop bandwidth (making them easy to filter), while staying low enough that the loop gain and circuit impedance remain reasonable and controllable.

# Typical Performance

## Frequency Stability

This is a plot of MVR Allan Deviation over a five-day run. Think of this as representing the likely amount of frequency wander between measurements taken at varying intervals from 0.1 to 100,000 seconds.

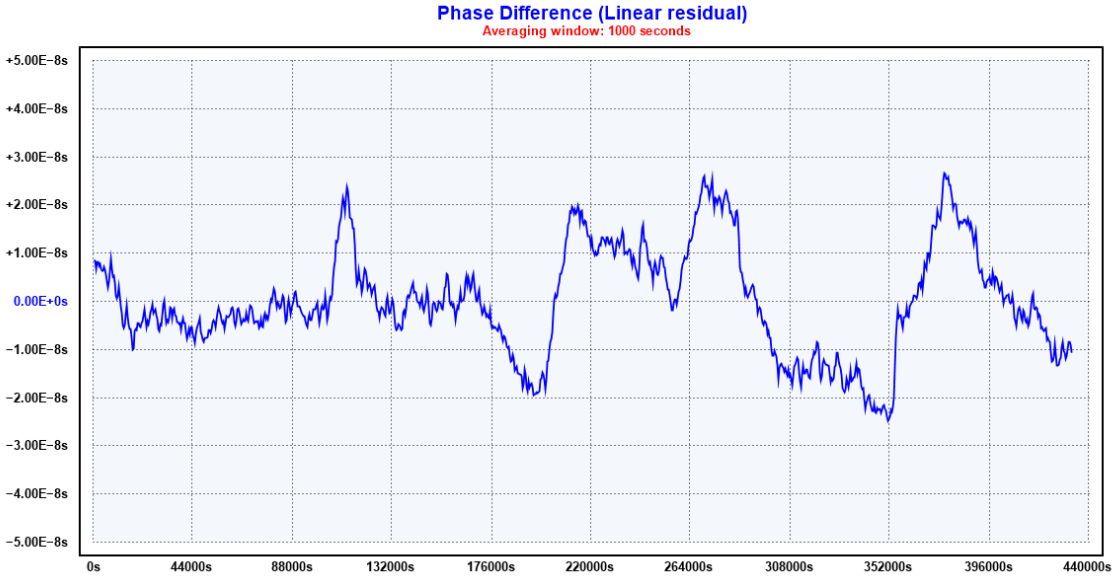



| Tau | Sigma(Tau) |
|---|---|
| 1s | 9.09E−10 |
| 2s | 8.39E−10 |
| 4s | 6.50E−10 |
| 8s | 4.09E−10 |
| 10s | 3.46E−10 |
| 20s | 2.40E−10 |
| 40s | 1.75E−10 |
| 80s | 1.07E−10 |
| 100s | 8.66E−11 |
| 200s | 4.41E−11 |
| 400s | 2.26E−11 |
| 800s | 1.12E−11 |
| 1000s | 9.03E−12 |
| 2000s | 4.63E−12 |
| 4000s | 2.44E−12 |
| 8000s | 1.43E−12 |
| 10000s | 1.24E−12 |
| 20000s | 9.16E−13 |
| 40000s | 5.70E−13 |
| 80000s | 3.63E−13 |
| 100000s | 2.06E−13 |

| Trace | Notes | Input Freq | Input Amplitude | Ref Freq | ENBW | Duration | Elapsed | Instrument |
|---|---|---|---|---|---|---|---|---|
| MVR A2 | vs. HP5071A | 10 MHz | 14.5 dBm | 10 MHz | 5 Hz | 5d 0h 0m 0s | 5d 0h 0m 0s | TimePod 5330A |

Stability of about $9 \times 10^{-10}$ at one second means that a 10 MHz output will wander by about 9 milliHertz from second to second. At longer averaging intervals the noise becomes lower as the GPS frequency stability overtakes that of the internal oscillator.

The GPS signal has both short term noise and its own longer-term wander. The plot below shows a plot of the signal phase over five days.



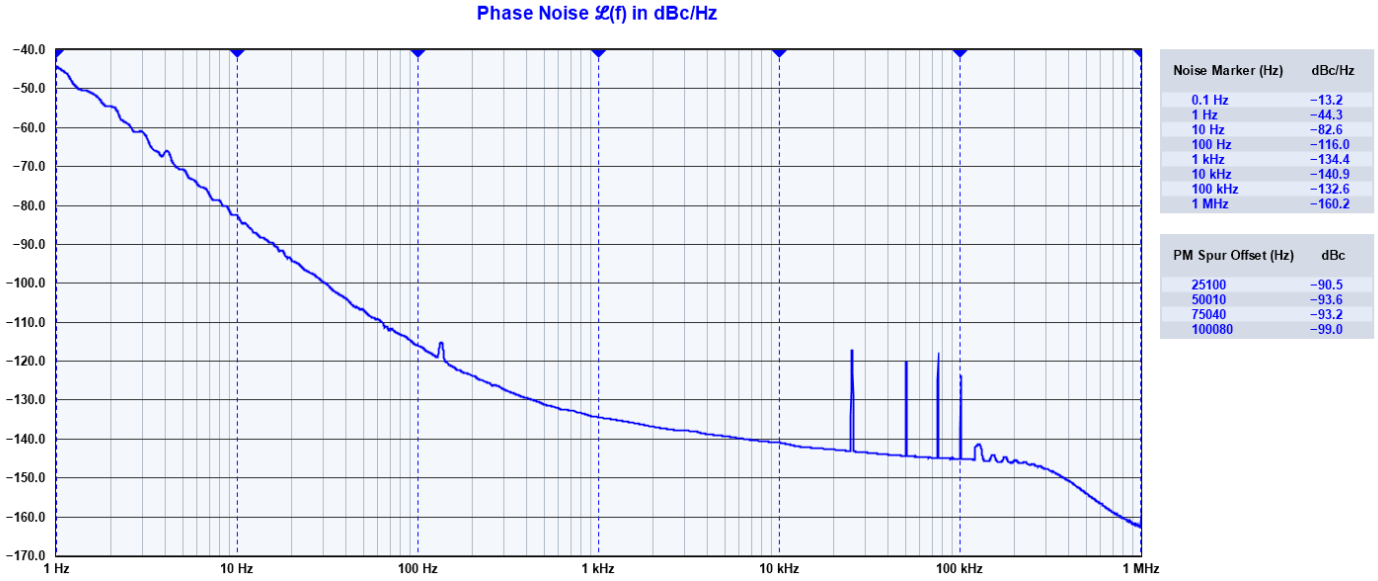

| Origin | Slope (sec/sec) |
|---|---|
| −5.44E−9 | +2.70E−13 |

| Trace | Notes | Input Freq | Input Amplitude | Ref Freq | ENBW | Duration | Elapsed | Instrument |
|---|---|---|---|---|---|---|---|---|
| MVR A2 | vs. HP5071A | 10 MHz | 14.5 dBm | 10 MHz | 5 Hz | 5d 0h 0m 0s | 5d 0h 0m 0s | TimePod 5330A |

The phase varies by about 12 nanoseconds RMS and 86 nanoseconds peak-to-peak over the period. (Note that in this plot the slight frequency offset of the reference oscillator, a Cesium frequency standard known to be offset from GPS by about $-3x10^{-13}$ has been removed.)

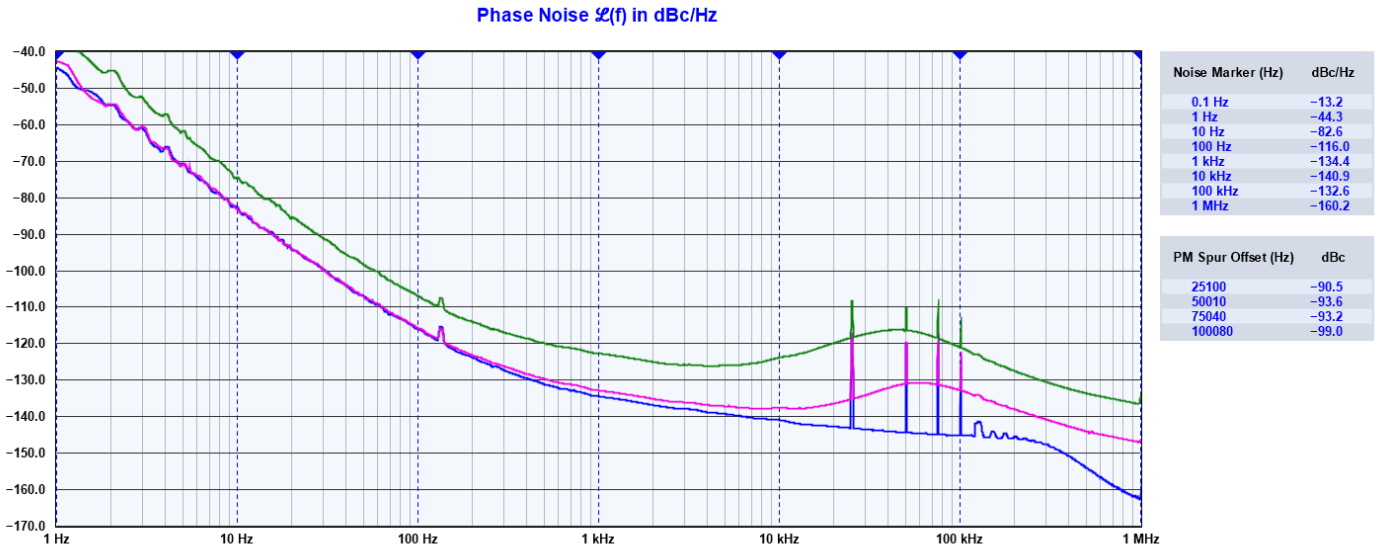
## Phase Noise

The plot below shows the phase noise of the 10 MHz output, which is derived directly from the VCXO. The spurs at 25, 50, 75, and 100 kHz are bleed-through of the PLL comparison frequency. These spurs are all more than 110 dB below the carrier. (Note: this plot was made using a prototype board. The production board has some layout changes that might lower the levels of these spurs somewhat.)




| Trace | Notes | Input Freq | Input Amplitude | Ref Freq | ENBW | Duration | Elapsed | Instrument |
|---|---|---|---|---|---|---|---|---|
| MVR A2 Direct 10 MHz | vs. ULN | 10 MHz | 14 dBm | 10 MHz | 5 Hz | 15m 0s | 15m 0s | Microchip 53100A |

The output via the Si5351A synthesizer chip has higher phase noise, particularly at large carrier offsets, than the direct signal. The plot below shows the phase noise of the synthesizer at both 10 MHz and 27 MHz, along with the direct output as shown above:




| Trace | Notes | Input Freq | Input Amplitude | Ref Freq | ENBW | Duration | Elapsed | Instrument |
|---|---|---|---|---|---|---|---|---|
| MVR A2 Direct 10 MHz | vs. ULN | 10 MHz | 14 dBm | 10 MHz | 5 Hz | 15m 0s | 15m 0s | Microchip 53100A |
| MVR A2 CLK0 10 MHz v24.1 | vs. ULN | 10 MHz | 12 dBm | 10 MHz | 5 Hz | 15m 0s | 15m 0s | Microchip 53100A |
| MVR A2 CLK1 27 MHz v24.1 | vs. ULN | 27 MHz | 9 dBm | 10 MHz | 5 Hz | 15m 0s | 15m 0s | Microchip 53100A |

The 27 MHz noise curve is higher than at 10 MHz primarily due to scaling; phase noise increases 20log(N) with carrier frequency so all else being equal, the noise at a given offset from the carrier at 27 MHz would be about 8.6 dB higher than at 10 MHz. This plot largely follows that.

# Assembly

The MVR is provided with nearly all components already installed, but there are three parts that must be soldered to the circuit board. Your package should include the following components:

- MVR circuit board
- Enclosure with blank end plates and 8 screws
- Front and rear end plates
- Five sets nuts and washers for SMA connectors
- One 10 uF film capacitors
- One 4.7 Megohm resistor
- One XIAO processor module with two 7-pin headers
- One piece of pink insulating foam
- One piece of adhesive transfer tape

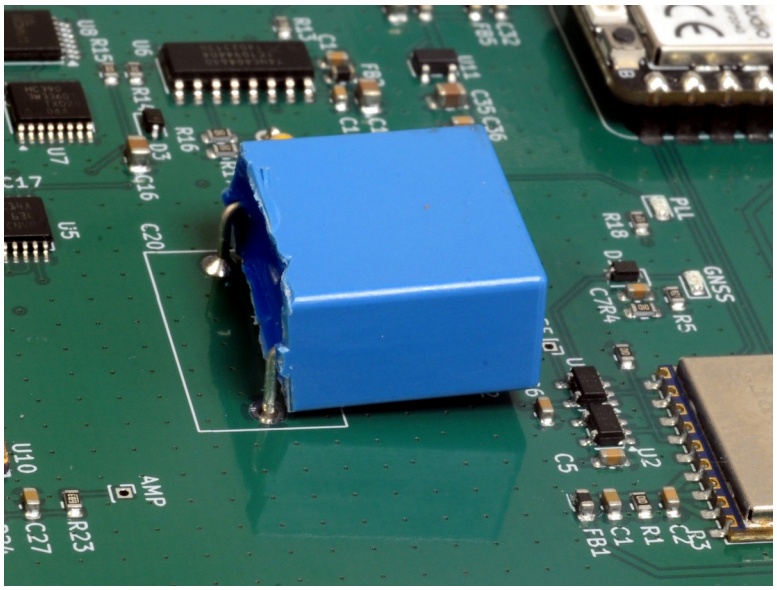
It's easiest to install these parts in this order, from thinnest to thickest:

1. Install the 4.7 megohm resistor at R19.

2. The 10 uF film capacitor is installed at C20.

   **READ THE FOLLOWING BEFORE MOUNTING IT!!!**

   This part is a little too tall to fit into the enclosure. Before soldering it, bend the two leads 90 degrees against the bottom so they extend just beyond the long side of the body. Place the capacitor facing the back of the board (toward the silver GNSS module). There will be just enough lead length to fit the leads through the two holes and solder them. The capacitor body will be at an angle relative to the PCB, but will lie flat enough to fit into the enclosure. See the picture below.




   **NOTE:** Make sure the capacitor is folded down toward the **BACK** of the PCB, not the front. Do not try to mount it flat in that nice open space; you will be using that space later.

3. The two seven-pin header strips mount on the bottom of the XIAO module, with the long pins facing down. Solder the short pins on the top of the module.

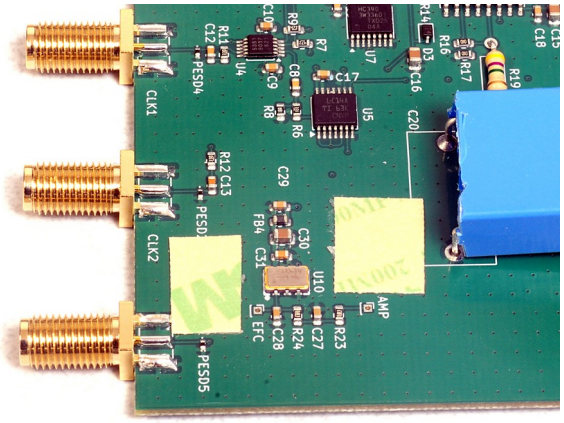
4. Then, insert the module with the USB connector facing the rear of the circuit board into the two rows of seven holes on the PCB, ensuring that the header is flush to the board. Solder the 14 pins to the bottom of the PCB. Trim the long ends.

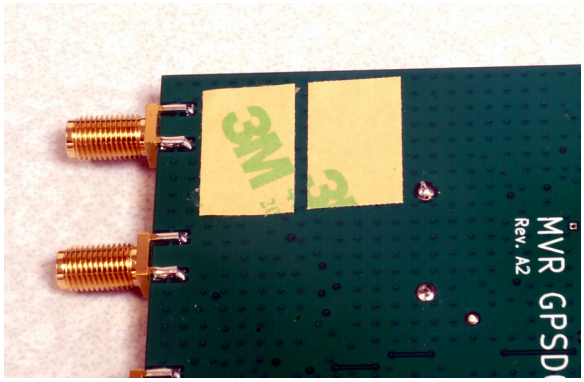
That's all the soldering there is to do, but there is one more very important bit of assembly. The VCXO needs to have as much thermal isolation as possible. A piece of pink foam and some adhesive transfer tape is provided to insulate the area above and below the VCXO on the PCB. Together with the enclosure, this significantly improves the MVR's short term stability. Here's how to install the foam:

1. Cut the foam into two pieces – one about 25 by 25 mm, and the other about 35 by 35 mm.

2. Cut the transfer tape into four pieces, two about 8 by 12 mm for the top of the board, and two about 10 by 20 mm for the bottom. The sizes aren't critical, but should be large enough to cover most of the area indicated in the photos below, without touching any components on the top of the board.

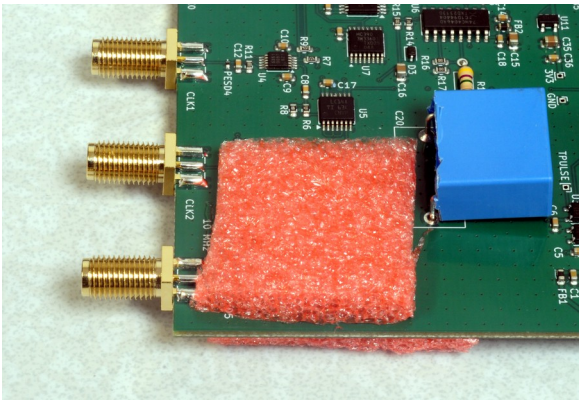
3. The transfer adhesive is weird stuff. It has two backing layers, one translucent and the other brown. After cutting to size, remove the translucent layer (a small tweezers or the edge of a knife blade may help in grabbing the corner) and then apply that side to the PCB, pressing very firmly and using your fingernail or a flat tool to squeegee it down. Apply all four pieces, two to each side of the board as shown below (the tape on the bottom should be underneath the VCXO area).

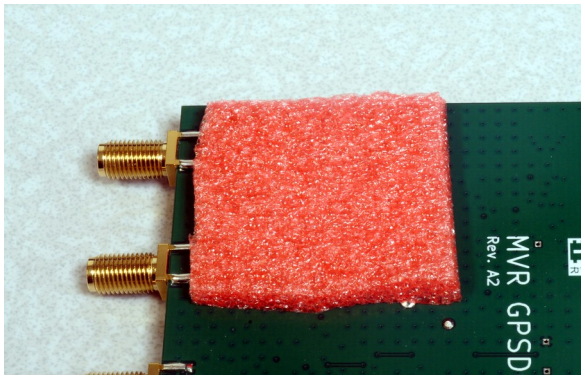



4. Carefully peel off the top paper from the adhesive on the top side of the board, using a knife blade or tweezers to grab a corner of the paper. When the paper comes off, the adhesive should be left on the board. Press the smaller (25 by 25 mm) piece of foam onto the top of the board, applying substantial pressure with your fingers. The foam should stick to the adhesive.

5. Similarly, peel off the top paper from the adhesive on the bottom of the board and apply the larger 35 by 35 mm piece of foam to the area underneath the VCXO.






6. Now you are ready to put the board into the enclosure. First, attach the front panel to one end of the enclosure using four of the screws provided. Note that the enclosure is positioned correctly when the interior slots are toward the bottom.

7. From the other end of the enclosure, slide the front end of the board (with four SMA connectors) into the guide slot. The board should go all the way in, and the SMA connectors should appear through the four holes in the front panel. If they don't fit cleanly, loosen the front panel screws, then re-tighten them after the board is in place.

8. Attach the rear panel using the other four screws.

9. Put the provided washers and nuts on the SMA connectors. Tighten them; finger tight is enough.

# Appendix A
# Firmware Load Using the Arduino IDE

You can also compile the firmware and upload it to the module using the Arduino IDE. This requires some configuration. After installing the Arduino IDE, do the following steps to prepare for using the Seeed XIAO RP2040 module:

1. Go to **File/Preferences/Additional Board Manager URLs**, then paste:
   **https://github.com/earlephilhower/arduino-pico/releases/download/global/package_rp2040_index.json**
   into the text box.

2. Go to **Tools/Boards/Board Manager** and search for **RP2040** and select **Raspberry Pi Pico/RP2040/RP2350** by Earle F. Philhower, III. Don't use Arduino's own "Mbed OS RP2040" package.

3. Go to **Sketch/Include Library/Manage Libraries**, search for **NeoPixel** and install **Adafrult/NeoPixel**.

4. Select the board by going to
   **Tools/Board/Raspberry Pi Pico/RP2040/RP2350/Seeed XIAO RP2040**

5. Select the Com Port if necessary.

6. If using Linux, you'll need to set permissions so your user ID can access the serial port. Issue this at the command line:
   **sudo usermod -aG dialout $USER**
   Then log out of that session and log back in again.

7. If you have git installed, do
   **git clone https://github.com/TAPR/mvr_firmware.git**
   in the Arduino sketchbook folder. Otherwise, download the zip file and unzip it in the folder.

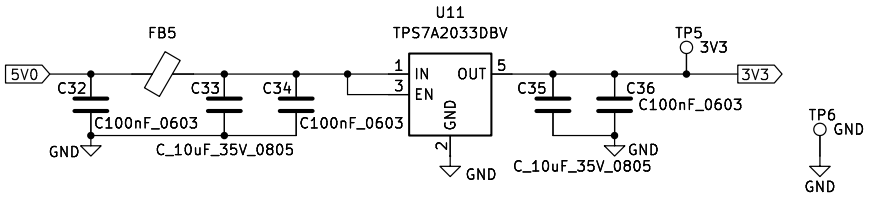
8. Point the IDE to the **sketchbook/mvr_firmware** folder, then click on mvr_firmware.ino to open the sketch.

You should now be able to edit, build, and upload the firmware to the MVR.

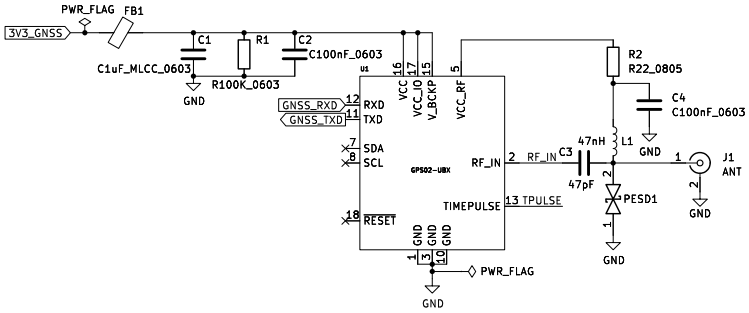
# Appendix B

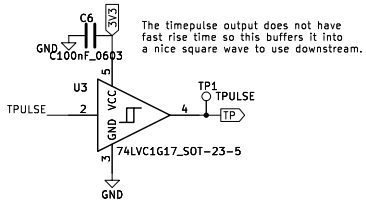
# Schematics and Layout

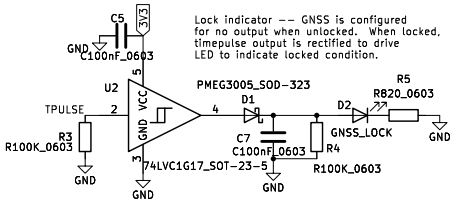
NOTE:
A higher-quality schematic package is available at
https://github.com/TAPR/mvr_firmware/docs/mvr_rev_B_schematics.pdf

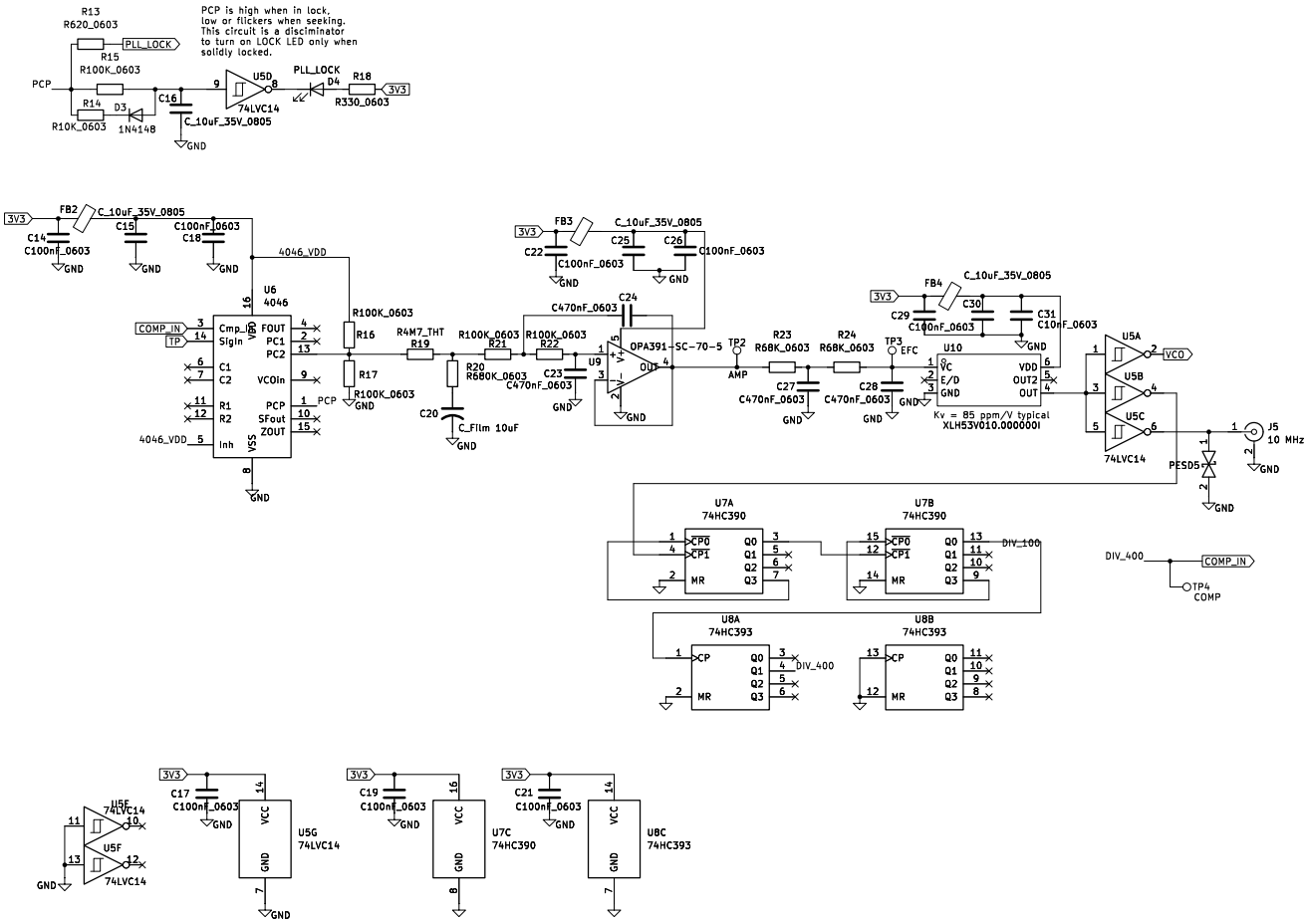


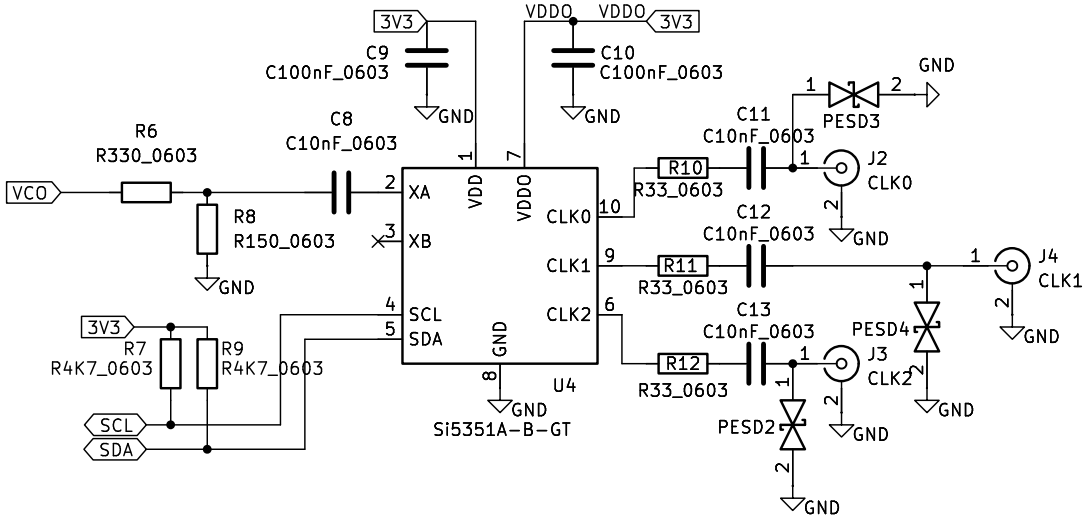

MS621FE -- 728-1134-ND

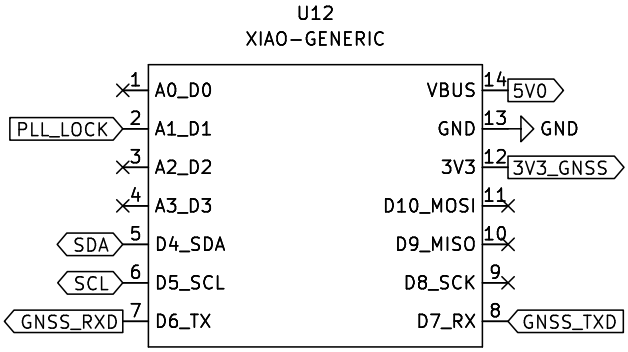


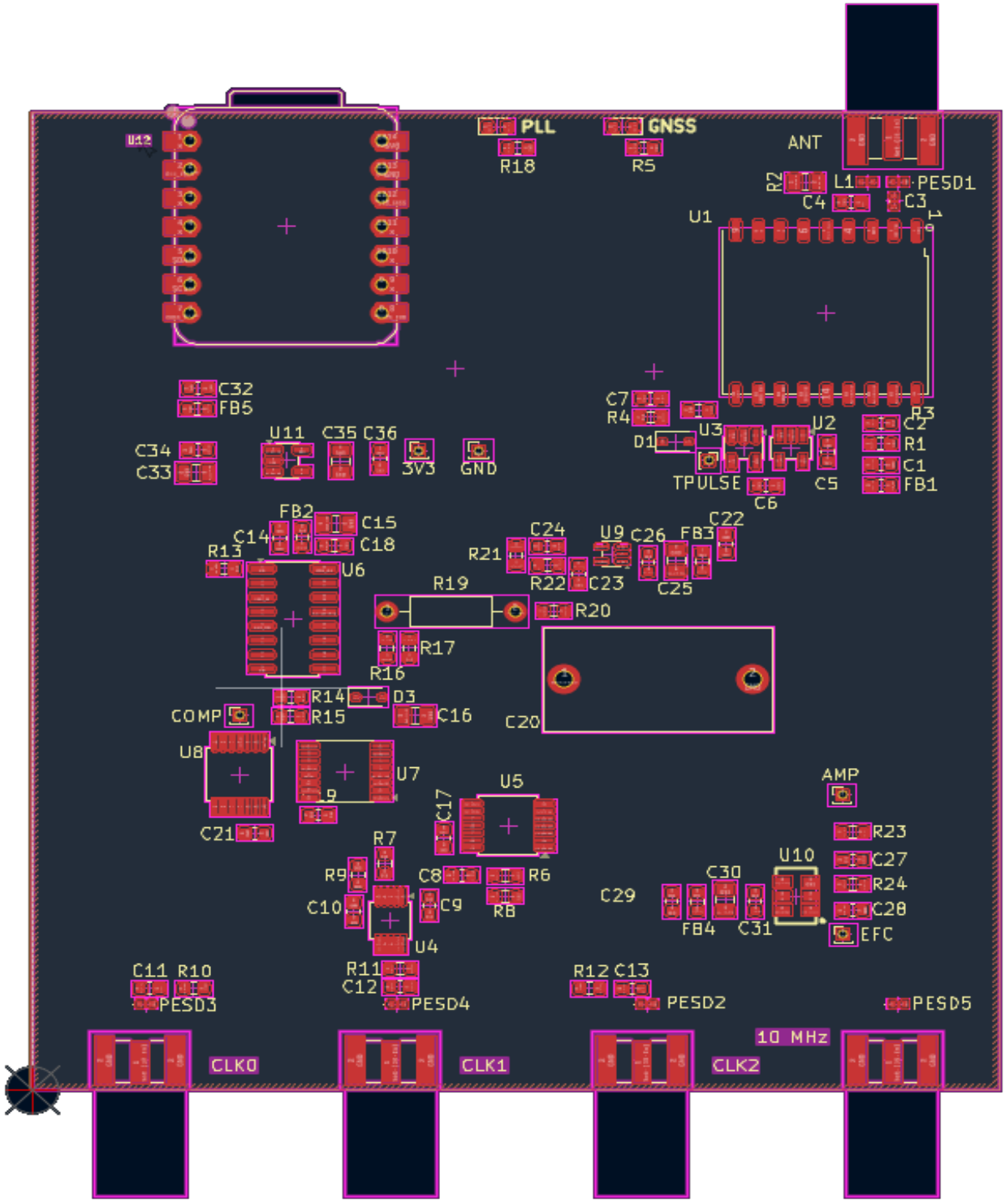




PCP is high when in lock, low or flickers when seeking. This circuit is a discriminator to turn on LOCK LED only when solidly locked.
R13
R620_0603
PLL_LOCK
R15
R100K_0603
PCP
R14
R10K_0603
D3
1N4148
C16
C_10uF_35V_0805
U5D
74LVC14
PLL_LOCK
D4
R18
R330_0603
3V3
GND
FB2
C_10uF_35V_0805
C14
C100nF_0603
C15
C18
4046_VDD
U6
4046
COMP_IN
TP
Cmp_In
SigIn
VDD
FOUT
PC1
PC2
C1
C2
VCOin
R1
R2
PCP
SFout
ZOUT
Inh
VSS
R100K_0603
R16
R17
R4M7_THT
R19
R20
R680K_0603
C20
C_Film 10uF
FB3
C22
C25
C26
C24
C470nF_0603
R21
R22
C23
U9
OPA391-SC-70-5
TP2
AMP
R23
R68K_0603
R24
C27
C28
TP3
EFC
U10
I²C
E/D
VDD
OUT2
OUT
FB4
C29
C30
C31
C10nF_0603
U5A
U5B
U5C
74LVC14
VCO
Kv = 85 ppm/V typical
XLH53V010.000000I
PESD5
J5
10 MHz
U7A
74HC390
U7B
74HC390
DIV_100
DIV_400
COMP_IN
TP4
COMP
U8A
74HC393
U8B
74HC393
U5F
74LVC14
U5G
74LVC14
U7C
74HC390
U8C
74HC393


3V3
C9
C100nF_0603
GND
VDDO
VDDO
3V3
C10
C100nF_0603
GND
GND
1
2
PESD3
C11
C10nF_0603
R6
R330_0603
C8
C10nF_0603
R10
R33_0603
J2
CLK0
VCO
2
XA
VDD
VDDO
CLK0
10
1
7
1
2
GND
R8
R150_0603
3
XB
C12
C10nF_0603
CLK1
9
R11
R33_0603
1
J4
CLK1
GND
4
SCL
CLK2
6
C13
C10nF_0603
PESD4
3V3
R7
R4K7_0603
R9
R4K7_0603
5
SDA
GND
R12
R33_0603
J3
CLK2
8
U4
GND
Si5351A-B-GT
PESD2
SCL
SDA
GND
GND
GND
GND


U12
XIAO-GENERIC
1
A0_D0
PLL_LOCK
2
A1_D1
3
A2_D2
4
A3_D3
SDA
5
D4_SDA
SCL
6
D5_SCL
GNSS_RXD
7
D6_TX
VBUS
14
5V0
GND
13
GND
3V3
12
3V3_GNSS
D10_MOSI
11
D9_MISO
10
D8_SCK
9
D7_RX
8
GNSS_TXD


U12
PLL
R18
GNSS
R5
ANT
R2
L1
PESD1
C4
C3
U1
C32
FB5
C34
C33
U11
C35
C36
3V3
GND
C7
R4
D1
U3
U2
R3
C2
R1
C1
FB1
TPULSE
C6
C5
FB2
C14
C15
C18
R13
U6
C24
R21
R22
C23
U9
C26
FB3
C22
C25
R19
R20
R17
R16
R14
R15
D3
C16
COMP
C20
U8
U7
C21
U5
C17
AMP
R23
C27
R24
C28
EFC
U10
C30
C29
FB4
C31
R7
R9
C8
R6
C10
C9
R8
U4
C11
R10
PESD3
R11
C12
PESD4
R12
C13
PESD2
PESD5
10 MHz
CLK0
CLK1
CLK2